Chaos Testing Script in a Self-Healing Order Tracker

The Chaos Testing Script is an important component of the Self-Healing Order Tracker project. Its purpose is to simulate software failures and evaluate whether the application can detect problems, apply suitable recovery mechanisms, and verify the results.

The system tests five main scenarios: worker failure, temporary network errors, tracking-provider unavailability, cached fallback, and invalid order data.

When a failure is introduced, the application detects the issue and selects an appropriate recovery mechanism. A watchdog restarts the simulated worker, exponential backoff retries temporary failures, a circuit breaker limits repeated calls to an unavailable provider, cached fallback maintains access to the last known order status, and data repair restores invalid order information from available recovery data.

The application records incidents and recovery information so that its behavior can be examined. These records also support the monitoring and analytics features of the project.

The main objective is to improve application resilience, reduce unnecessary service interruptions, and maintain access to reliable order information whenever possible.

The current implementation uses controlled, in-process failure simulations. It demonstrates important self-healing principles but does not guarantee recovery from every possible software, hardware, database, or network failure. Further improvements would include persistent recovery snapshots, integration testing against real services, external process supervision, and stronger recovery verification.

Final conclusion

Your Chaos Script is not just a program that creates errors. It is a **testing mechanism for evaluating how your Self-Healing Order Tracker responds to failures**.

Its overall workflow is:

Inject Failure → Detect Problem → Diagnose Cause → Apply Recovery → Verify Result → Record Incident

The most important distinction to remember is that your current project demonstrates controlled simulations of failures. The next step is to make those tests more realistic and

ensure that recovery works reliably even when the application or database encounters genuine failures.